# Distributed Systems: Project 4 Task 2

MovieBrowser Android App

Name - Kavi Sarna
AndrewId - ksarna

1. **Android Application:**

This application is aimed at helping people find details about a movie they would like to search for. They would just need to provide a name or part of the name of the movie and it will fetch a list of all the movies that have that string as part of their title. From the list, a user can select one movie to know more about it.

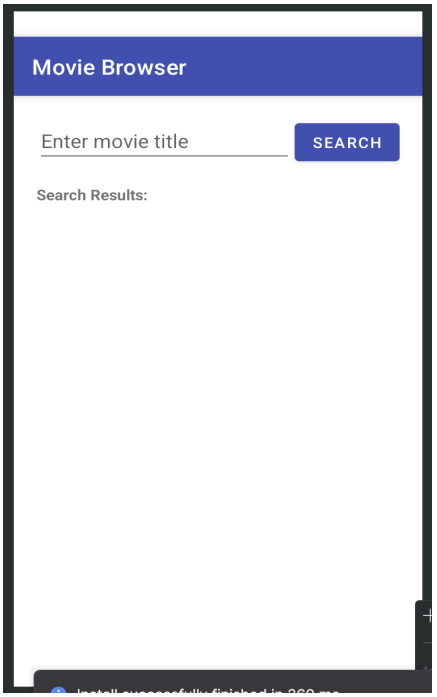
A. **View components**:

The app uses the following view components:

i. *TextView* - To display movie title, release date, ratings, overview, etc
ii. *ListView* - To display the list of the movies
iii. *EditText* - To capture the user input
iv. *ImageView* - To show the poster of the movie

B. **Input from the user**:

The application asks for user input using a text area where a user can enter a query to find the list of movies that contain what the user is looking for in their title.



C. **HTTP Request**:

The application makes 2 HTTP requests to the TMDB service running in GitHub codespaces.
It makes one request to search the movies at the endpoint - /api/search/movies?query=”movie”
It makes one request to get the details of the movie at the endpoint - /api/movies/58

The requests are run on the BackgroundTask thread using the “java.net.HttpUrlConnection” and an InputStream.

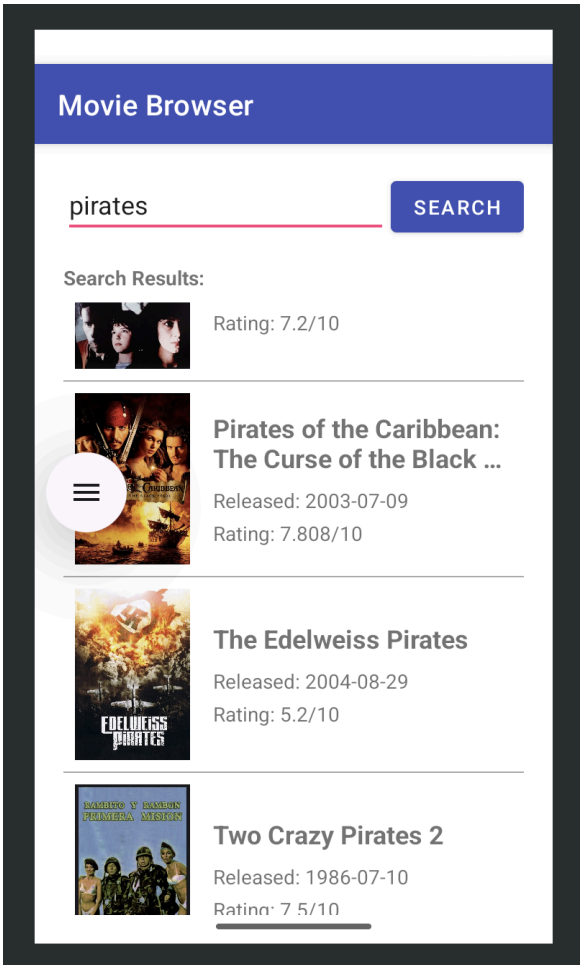
D. **JSON Response:**

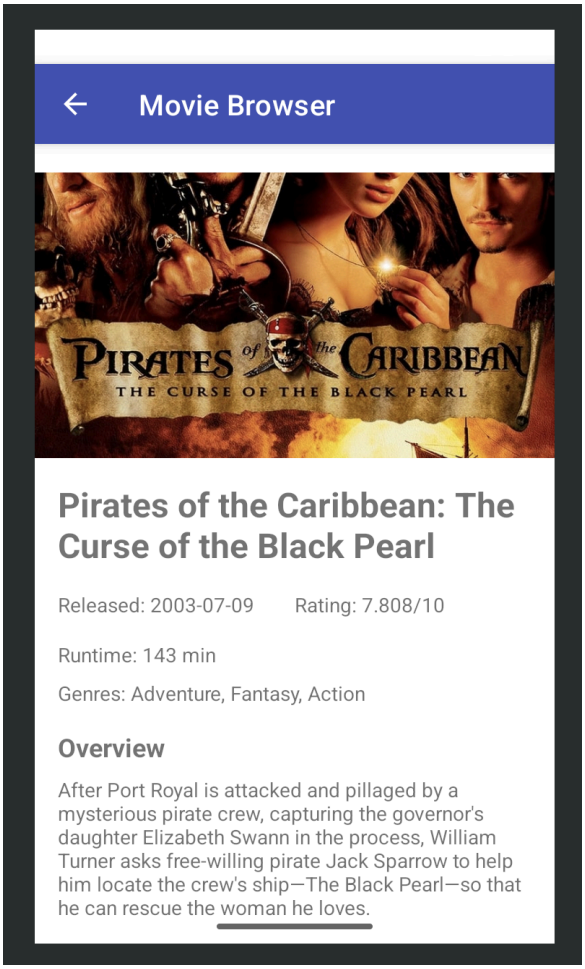
The app parses a JSON response from the TMDB service.
The search request response includes a poster, title, release date, and rating.
The details request response includes a poster, title, overview, release date, rating, runtime, and genres.

It parses the JSON response using the org.json.JSONObject to convert the data from string to object format. This helps properly handle different data formats including Strings, integers, and date.

E. **New Information for the user**:

The app displays the new information to the user based on the input that the user has provided.
It creates a list of movies which shows the user a scrollable view of the movies related to their search query. It also allows the user to click on a single movie in the list and see more details about it.



F. **Repeatable**:
The app allows the user to perform multiple operations without restarting the app. If a user wants to find details of another movie after they have done with the one they were looking for they just need to go back using the back arrow at the top left corner and then search for another movie.

2. **Web Service:**

The web service is a java based web application which connects to the TMDB service and fetches data from it to pass on to the Android App. It also takes care of handling any errors encountered with the requests and logs all the requests to a mongodb collection.

A. **API Implementation**:
The service implements 3 Restful HTTP APIs which receives request from the Android App and the dashboard interface using 3 separate servlets. The requests use different query and path params to take the inputs from the Android app and use them to pass them to the TMDB APIs to get the data, and then it logs the request along with the response.
The implemented endpoints are -
/api/search/movies
/api/movies/{movieId}
/dashboard

B. **HTTP Request from Android App**:
The service receives 2 different GET requests from the Android App and returns with a JSON response.
The JSON response consists of a list of movies in case of a search request, for movie details request it just returns the details of that movie as a JSON object.

C. **Business Logic:**
The service implements a very simple business logic where it receives a request from the Android App, parses it and makes an API call to the TMDB APIs depending upon the type of request received.

D. **Response Formatting:**
It takes care of sending only the details that are needed by the Android application and not send all the details that it receives from the TMDB API.
The response for the search request consists of a list of movies where each movie has the following schema.

```
public class Movie {  7 usages   Kavi Sarna
    private int id;  4 usages
    private String title;  4 usages
    private String releaseDate;  4 usages
    private String posterPath;  3 usages
    private double rating;  4 usages
```

The response for the movie details request consists of the details about a movie and consists of the following schema.

```
public class MovieDetail {  6 usages   Kavi Sarna
    private int id;  4 usages
    private String title;  4 usages
    private String overview;  3 usages
    private String releaseDate;  4 usages
    private double rating;  4 usages
    private String posterPath;  3 usages
    private List<String> genres;  4 usages
    private int runtime;  4 usages
```

3. **Handle Error Conditions:**

The webservice takes care of handling the errors from the user/android app by making sure the user is calling the proper endpoint or if the user is sending the proper inputs needed for the TMDB API.

```
// Validate path
if (pathInfo == null || pathInfo.equals("/")) {
    response.setStatus(HttpServletResponse.SC_BAD_REQUEST);
    JsonObject error = new JsonObject();
    error.addProperty( property: "error",  value: "Missing movie ID");
    out.print(gson.toJson(error));

    // Log error
    logEntry.setStatusCode(HttpServletResponse.SC_BAD_REQUEST);
    logEntry.setResponseTime(System.currentTimeMillis() - startTime);
    logEntry.setResponseSize(error.toString().length());
    loggingService.logRequest(logEntry);

    return;
}

// Extract movie ID from path
String movieIdStr = pathInfo.substring( beginIndex: 1);
int movieId;

// Validate movie ID
try {
    movieId = Integer.parseInt(movieIdStr);

    logEntry.setQueryParams(movieIdStr);
} catch (NumberFormatException e) {
    response.setStatus(HttpServletResponse.SC_BAD_REQUEST);
    JsonObject error = new JsonObject();
    error.addProperty( property: "error",  value: "Invalid movie ID format");
    out.print(gson.toJson(error));

    // Log error
    logEntry.setStatusCode(HttpServletResponse.SC_BAD_REQUEST);
```

It also takes care of handling errors with response from the TMDB API.

```
// Get response code
int responseCode = connection.getResponseCode();
String responseMessage = connection.getResponseMessage();

if (responseCode == HttpURLConnection.HTTP_NOT_FOUND) {
    throw new IOException(responseMessage);

} else if (responseCode != HttpURLConnection.HTTP_OK) {
    throw new IOException("HTTP error code: " + responseCode);
}
```

The android app also handles the API response errors from the web service in the similar way using the HTTP response codes.

4. **Log Useful Information:**

For logging the requests that are coming to the logging service, it uses the MongoDB database to store the logs. It captures the client device where the request came from, the timestamp at which the request came, the endpoint, query params, response time and the response too.

```
public class LogEntry {  14 usages   Kavi Sarna
    private String id;                // MongoDB document ID  3 usages
    private Date timestamp;           // When the request was received  4 usages
    private String endpoint;          // API endpoint called  3 usages
    private String requestMethod;     // HTTP method (GET, POST, etc.)  3 usages
    private String queryParams;       // Query parameters in the request  3 usages
    private String userAgent;         // User agent (includes device model)  3 usages
    private String clientIp;          // Client IP address  3 usages
    private long responseTime;        // Response time in milliseconds  3 usages
    private int statusCode;           // HTTP status code of response  3 usages
    private String tmdbEndpoint;      // TMDB API endpoint called  3 usages
    private int tmdbResponseCode;     // Response code from TMDB API  3 usages
    private long tmdbResponseTime;    // Response time from TMDB API in milliseconds  3 usages
    private String tmdbResponseJSON;   // Response json from TMDB API  2 usages
    private int responseSize;         // Size of response in bytes  3 usages
```

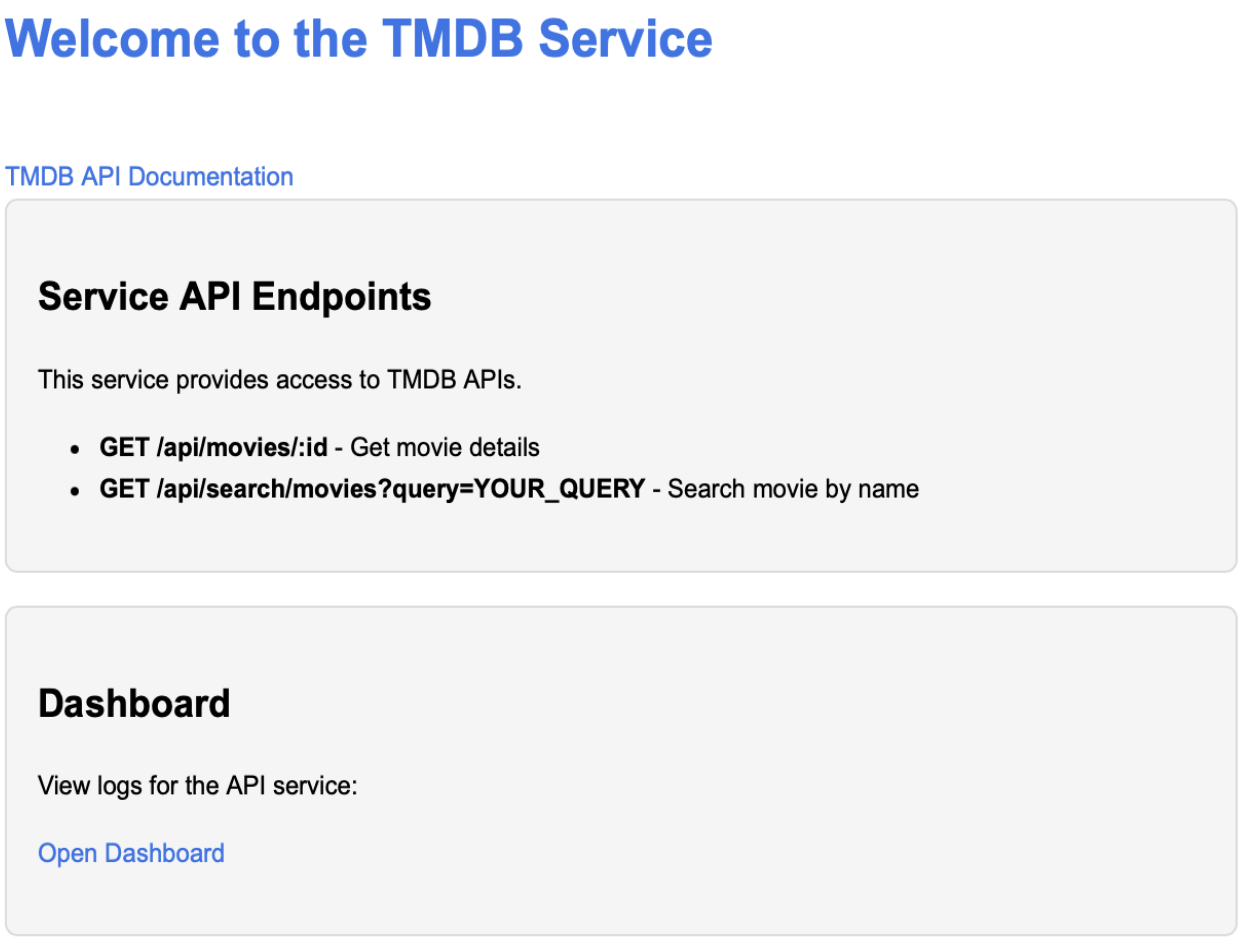
5. **Log Information in the Database:**

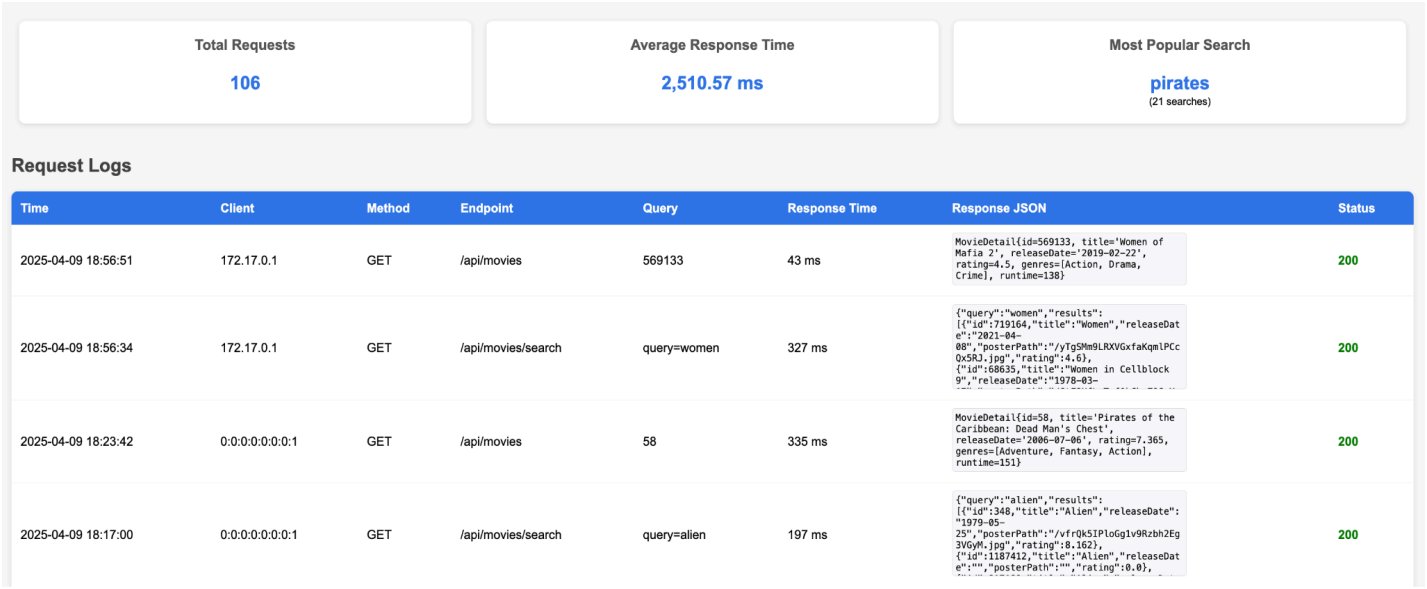
It logs all the information about the above mentioned logs in a MongoDB collection on Mongo Atlas cluster.

```
_id: ObjectId('67f6c2f307dcc01dc25ddc9d')
timestamp : 2025-04-09T18:56:51.583+00:00
endpoint : "/api/movies"
requestMethod : "GET"
queryParams : "569133"
userAgent : "Dalvik/2.1.0 (Linux; U; Android 16; sdk_gphone64_arm64 Build/BP22.2502…"
clientIp : "172.17.0.1"
responseTime : 43
statusCode : 200
tmdbEndpoint : "/movie/569133"
tmdbResponseCode : 200
tmdbResponseTime : 32
tmdbResponseJSON : "MovieDetail{id=569133, title='Women of Mafia 2', releaseDate='2019-02-…"
responseSize : 313
```


6. **Operation Anlaytics and Dashboard:**

   It generates a dashboard where the logs from the database can be visualized.
   It also captures 3 operation analytics - total number of requests, average response time, and most popular search term.
   To reach the dashboard we can click on the dashboard link on the landing page of the web service.






## 7. Deployment with Codespaces:

The repository for my project is:
https://github.com/CMU-Heinz-95702/distributed-systems-project-04-Kavi-Sarna

The repository has a ROOT.war file which is a war file containing the web service.
It also has a Dockerfile to create the docker image for the service.
It has a devcontiner.json file which is used by codespaces to define the build for the docker image.

To deploy the service in codespaces, create a new codespace and choose the repository, it will open a vs code terminal inside the codespace.
Execute the following commands:

*docker build -t tmdb_service:1.0.0 .*
*docker images*
*docker run -it -p 8080:8080 tmdb_service*

Go to the ports section in the terminal and change the visibility of the port from *private* to *public*.

Copy the url and paste it in the browser to see the welcome screen for the web service.

Copy the same url of the codespace and change it in the HttpHandler.java file at line no 16.
Do remember to remove the trailing slash(/) at the end of the url.

```
2 usages
14  public class HttpHandler {
15
        2 usages
16      private static final String BASE_URL = "https://expert-space-parakeet-wwqjg5vwwr635qwr-8080.app.github.dev";
17
18
```